

SLIDE 1 — Title (~15 sec)

"Good morning sir and everyone. Thank you for joining us today I am Tushar Barua. My project is MediScript-E — a digital healthcare platform already live at [mediscript-e.vercel.app](#)."

SLIDE 2 — Abstract (~20 sec)

"MediScript-E is a platform where patients can manage their health digitally, doctors can handle appointments and prescriptions online, and admins can oversee the entire system.

On the right you can see the patient dashboard — this is what a real user sees after logging in.

The whole project was built following the SDLC methodology, using Next.js, TypeScript, and PostgreSQL."

SLIDE 3 — Outline (~15 sec)

"This presentation follows IEEE format — covering Related Work, all SDLC phases from Requirements to Deployment, and finishing with Challenges and Conclusion."

SLIDE 4 — Related Work (~30 sec)

"If we look at this table, established platforms handle individual features really well, but they mostly operate in silos.

What's currently missing is a unified, secure ecosystem, and that's exactly the space MediScript-E is designed for."

SLIDE 5 — Problem Statement (~20 sec)

"The core problem is simple.

Prescriptions get lost, appointments need phone calls, medical records are scattered across hospitals, patients forget their medicine schedules, and there is no secure, role-based way to manage health data.

Healthcare is fragmented — and that is what this project solves."

SLIDE 6 — Proposed Solution (~25 sec)

"The solution is one platform with three roles.

Patients can book appointments, view prescriptions, set reminders, and upload medical records.

Doctors can manage appointments and issue digital prescriptions.

Admins can monitor and manage the whole platform.

The system has 25 plus API endpoints, 9 database models, 3 user roles, 5 core features — and it is 100 percent live on Vercel."

SLIDE 7 — Requirements Analysis (~25 sec)

"In Phase 1, I defined both functional and non-functional requirements.

On the functional side — the system needs appointment booking, PDF prescriptions, medicine reminders, cloud storage, and role-based dashboards.

On the non-functional side — security was the top priority, followed by performance using serverless APIs, scalability with Vercel and Supabase, and a responsive, mobile-friendly user interface."

SLIDE 8 — Use Cases & Actors (~20 sec)

"The system has 5 actors — Patient, Doctor, Admin, a Cron System for automated tasks, and OAuth Providers for third-party login.

The use cases cover the full lifecycle — from registration to booking.

The appointment status goes through PENDING to CONFIRMED and then COMPLETED — or it can be CANCELLED at any point."

SLIDE 9 — System Architecture (~25 sec)

"For the architecture, I used a Three-Tier approach.

As shown here, it unifies a Next.js frontend, serverless API routes, and a Supabase database, supported by external services for AI and automated reminders."

SLIDE 10 — Database Design (~20 sec)

"The database has 9 models, 3 roles, and cascade delete throughout.

A User has either a DoctorProfile or a PatientProfile, each linking to their relevant data.

Three important design decisions — prescriptions can be soft-archived without permanent deletion, OTPs are cleared from the database right after verification, and cascade delete ensures no orphan records remain."

SLIDE 11 — Tech Stack (~15 sec)

"The full stack is — Next.js 16, TypeScript, Tailwind CSS, NextAuth, Prisma, PostgreSQL via Supabase, Supabase Storage, Nodemailer, Groq SDK — all deployed on Vercel."

SLIDE 12 — Authentication & Security (~35 sec)

"When it comes to Authentication and Security, we implemented a robust 6-layer protection flow.

It starts with email verification and bcrypt password hashing, followed by Two-Factor Authentication and OAuth login, while securing user sessions through JWT and enforcing strict Role-Based Access Control on every API route."

SLIDE 13 — Patient Features (~25 sec)

"Patients have 4 core features.

First, Appointment Booking—allowing users to select a doctor, pick a date and time, and confirm.

Next, E-Prescription—with options to view diagnoses, medications, and download it as a PDF.

Then, Medicine Reminders—where a set schedule triggers automated daily email alerts.

And finally, the Medical Vault—enabling seamless report uploads, stored securely in the cloud."

SLIDE 14 — Doctor & Admin Features (~20 sec)

"On the left you can see the Doctor dashboard. Doctors can confirm, cancel, or complete appointments. When they issue a prescription, the appointment is automatically completed. They can also edit, archive, and delete prescriptions.

On the right is the Admin dashboard. Admins see real-time platform statistics, can delete users, and monitor all appointments with contact messages."

SLIDE 15 — MediBot & Search (~20 sec)

"Two special features.

On the left is MediBot, a Llama 3.1-powered AI chatbot that is available on all pages.

On the right is our Global Search, which opens with Ctrl+K and provides role-aware results tailored to each user."

SLIDE 16 — Testing (~15 sec)

"I tested all 7 areas manually and through integration testing — authentication flows, appointment lifecycle, prescription operations, medicine reminder email delivery, medical vault, security checks, and edge cases.

Everything passed."

SLIDE 17 — Results & Evaluation (~20 sec)

"Every single requirement has been successfully met across all SDLC phases—delivering all security layers, core features, and AI modules fully live on Vercel."

SLIDE 18 — Deployment (~15 sec)

"Deployment is fully automated on Vercel. I push to GitHub — Vercel detects it, builds, and deploys instantly.

On the right you can see the actual Vercel deployment dashboard."

SLIDE 19 — Challenges & Solutions (~25 sec)

"Every real project has challenges. Here are the ones I faced—which we resolved by optimizing serverless connection pools, fixing database time zone mismatches, and refining our authentication and cascade flows."

SLIDE 20 — Future Enhancements (~10 sec)

"Short term — real-time notifications using WebSockets, and a doctor availability calendar. Medium term — video consultation using WebRTC, and a mobile app using React Native. Long term — an AI symptom checker and HIPAA compliance."

SLIDE 21 — Conclusion (~20 sec)

"By following the complete SDLC, MediScript-E delivers a secure, AI-powered healthcare platform live on Vercel, proving that a structured engineering approach is key to building real-world applications that work."

SLIDE 22 — Q&A (~10 sec)

"Thank you. The platform is live at mediscript-e.vercel.app.

I am happy to take any questions."

Likely Questions & Short Answers

Q: Why Next.js instead of separate frontend and backend? A: Next.js lets me build both the frontend and API in one codebase. It simplifies deployment on Vercel and is easier to maintain.

Q: How is MediScript-E different from Practo or Zocdoc? A: Those platforms each solve one problem. MediScript-E combines booking, prescriptions, reminders, vault, AI chatbot, and 2FA — all in one free, open platform.

Q: How secure is the 2FA? A: The OTP is generated on the server, expires in 10 minutes, and is cleared from the database after use. The session is only created after confirming the OTP is null — so no bypass is possible.

Q: How does the cron job work? A: GitHub Actions fires every 5 minutes, calls the API with a Bearer key, matches the current Bangladesh time within plus or minus 5 minutes, and sends the email via Nodemailer.

Q: Why JWT over database sessions? A: JWT is stateless — no database lookup needed per request. This is ideal for serverless functions. The user role is stored in the token for instant access control checks.

Q: Why Supabase? A: Supabase gives a managed PostgreSQL database with connection pooling, built-in file storage, and a dashboard — all for free. It removes infrastructure overhead completely.

Q: Is the platform HIPAA compliant? A: Not yet — it is listed as a long-term goal. But the platform already uses encryption, role-based access control, and secure storage, which align with HIPAA principles.